Linux Namespaces and cgroups as OS Primitives for Lightweight Virtualization

Architecture, Isolation Mechanisms, and Performance Evaluation

Srikanth Nimmagadda

Z&A Infotek Corporation, New Jersey, USA

2018

Overview

- Linux namespaces and cgroups enable **lightweight virtualization**
- Containers share host kernel while maintaining isolation
- Balance between hypervisor-based VMs and bare-metal deployment
- Foundation for Docker, LXC, and modern container technologies

Key Topics:

- Architecture and isolation mechanisms
- Resource management capabilities
- Performance comparison with VMs

Virtualization Evolution

Traditional Hypervisor-Based VMs

- Strong isolation with separate guest OS
- High resource consumption and startup latency

Container-Based Virtualization

- OS-level virtualization sharing host kernel
- Process-level isolation
- Faster deployment, lower resource usage
- Improved scalability

Driver: Modern microservices and cloud-native architectures

Virtualization Technologies Comparison

Type	Isolation	Overhead
Full Virtualization (VMware, KVM)	Strong Separate OS	High Resource-intensive
Paravirtualization	Moderate Guest OS modified	Medium Reduced overhead
Containers	Process-level Shared kernel	Minimal Fast startup

Trade-off: Security vs. Performance

Linux Namespaces: Resource Isolation

Seven Types of Namespaces:

1. **PID Namespace** - Isolates process IDs
2. **Mount Namespace** - Isolates filesystem mount points
3. **Network Namespace** - Creates isolated network stacks
4. **IPC Namespace** - Isolates interprocess communication
5. **UTS Namespace** - Isolates hostname/domain name
6. **User Namespace** - Separates user and group IDs
7. **Cgroup Namespace** - Isolates control group view

Each namespace type isolates a specific system aspect

Namespace Isolation Mechanism

How Namespaces Work

- Create separate instances of global kernel resources
- Resources visible only within namespace
- Processes perceive isolated environments

System Calls

- `clone()` - Create process with new namespaces
- `unshare()` - Detach from current namespaces
- `setns()` - Join existing namespace

Lifecycle: Namespace persists while processes exist

Control Groups (cgroups): Resource Management

Purpose

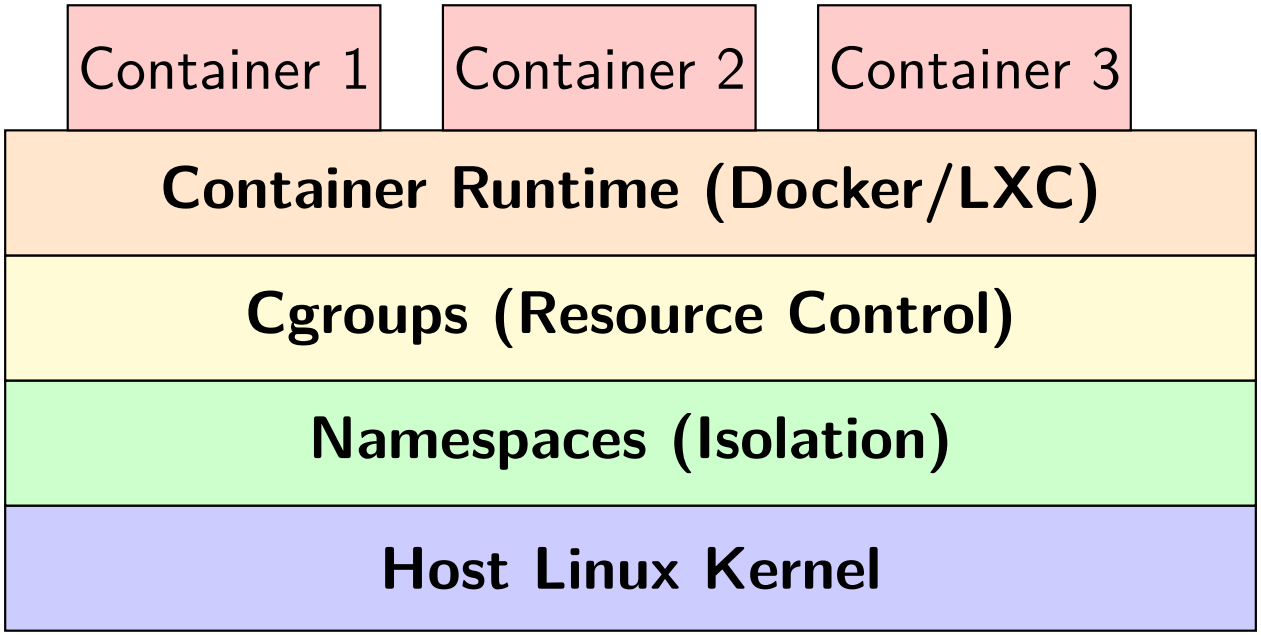
Hierarchical organization and management of processes with resource allocation control

Key Controllers:

- **CPU Controller** - Schedules CPU time
- **Memory Controller** - Limits and tracks memory usage
- **blkio Controller** - Regulates disk I/O throughput
- **Device Controller** - Restricts hardware access
- **Network Controller** - Manages bandwidth allocation

Enables fine-grained resource control and QoS guarantees

Container Architecture



Integration: Namespaces isolate views, cgroups control usage

Containers vs Virtual Machines

Containers:

- Share host kernel
- OS-level isolation
- Lightweight
- Fast startup (seconds)
- Lower overhead
- Weaker isolation

Virtual Machines:

- Separate guest OS
- Hardware-level isolation
- Resource-intensive
- Slow startup (minutes)
- Higher overhead
- Stronger isolation

Use Case: Containers for microservices, VMs for strict isolation

Performance Results: CPU & Memory

CPU Performance

Environment	Score	Overhead
Bare Metal	100%	0%
Container	96%	4%
VM (KVM)	85%	15%

Memory Performance

Environment	Speed	Overhead
Bare Metal	100%	0%
Container	97%	3%
VM (KVM)	80%	20%

Key Finding: Containers achieve near-native performance with minimal overhead

Performance Results: Network & Disk I/O

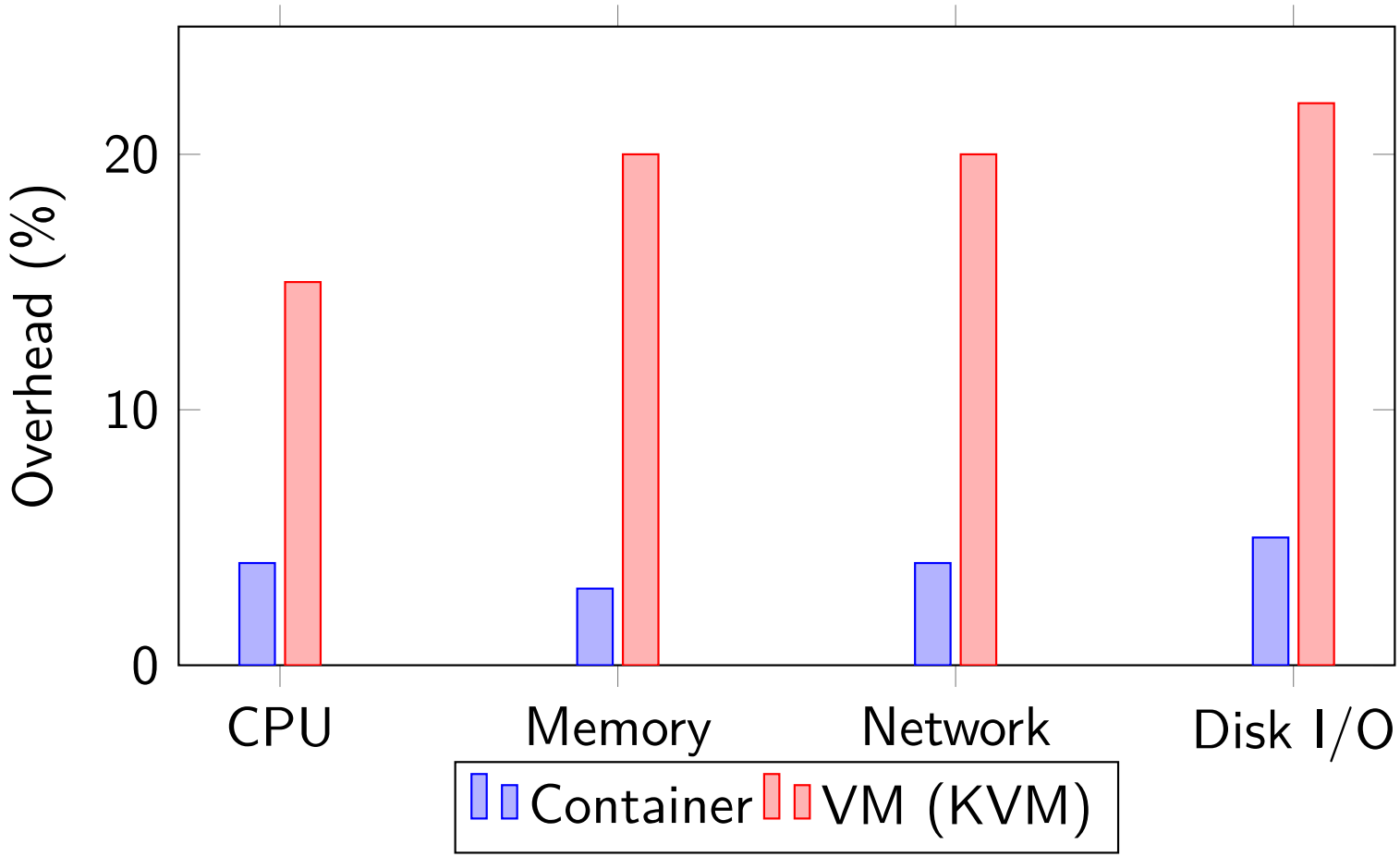
Network Performance

Environment	Throughput	T. Overhead	Latency	L. Overhead
Bare Metal	10 Gbps	0%	0.5 ms	0%
Container	9.6 Gbps	4%	0.55 ms	10%
VM (KVM)	8 Gbps	20%	0.75 ms	50%

Disk I/O Performance

Environment	IOPS	Overhead
Bare Metal	100%	0%
Container	95%	5%
VM (KVM)	78%	22%

Performance Overhead Comparison



Conclusion: Containers consistently outperform VMs across all metrics

Key Findings

Performance Advantages

- **Near-native performance:** <5% overhead in containers
- **Rapid startup:** Seconds vs minutes for VMs
- **Resource efficiency:** Minimal CPU, memory footprint
- **High density:** More containers per host

Architectural Benefits

- Kernel sharing eliminates OS duplication
- Direct kernel access reduces latency
- Fine-grained resource control via cgroups
- Flexible isolation via namespaces

Security and Isolation Trade-offs

Container Limitations

- Shared kernel = larger attack surface
- Kernel vulnerabilities affect all containers
- Weaker isolation boundaries than VMs
- Requires careful configuration

Mitigation Strategies

- Kernel hardening and security policies
- Runtime security monitoring
- User namespace mapping
- Complementary security frameworks (SELinux, AppArmor)

Decision: Balance performance needs with isolation requirements

Conclusion

- **Linux namespaces and cgroups** are fundamental primitives for lightweight virtualization
- Enable **near-native performance** with minimal overhead
- Foundation for **modern container ecosystems** (Docker, Kubernetes)
- Ideal for **microservices and cloud-native** applications

Future Directions

- Enhanced security mechanisms
- Advanced orchestration frameworks
- Integration with emerging technologies

Containers have revolutionized modern computing through efficient process isolation and resource management